Week 3: SELECT: querying one table

Start the SQLite shell

```
sqlite3 school.db
.tables               -- list tables
.schema students      -- show a table's definition
.headers on           -- show column names
.mode column          -- tidy output
.read file.sql        -- run a file of SQL
.quit
```

Lines starting with a dot are shell commands (no semicolon). **SQL statements end with** `;`.

The school database

```
students(student_id, name, class, gender, dob, email)
subjects(subject_id, name, teacher)
scores(student_id, subject_id, score)
ccas(cca_id, name, category, day)
student_cca(student_id, cca_id, role)
```

SELECT: the pattern

```
SELECT   columns
FROM     table
WHERE    condition on each row
ORDER BY column [ASC|DESC]
LIMIT    n;
```

`SELECT *` = every column. `SELECT DISTINCT class` = no repeats. `SELECT name AS student` renames a column in the output.

WHERE: conditions

Write	Meaning
`= != < > <= >=`	Compare. One `=` in SQL
`AND OR NOT`	Combine. Use brackets with mixed AND/OR
`BETWEEN 50 AND 70`	Inclusive range
`IN ('3A','3B')`	Matches any in the list
`LIKE 'T%'`	Pattern: `%` any characters, `_` one character
`IS NULL` / `IS NOT NULL`	Missing value test

```
SELECT name, class FROM students
WHERE class = '3A' AND name LIKE 'T%'
ORDER BY name;
```

NULL means "unknown"

- `score = NULL` **never matches**, not even NULL. Use `score IS NULL`.
- Any arithmetic or comparison with NULL gives NULL (not true). Rows where the condition is NULL are left out.
- SQLite sorts NULL **first** in `ORDER BY ... ASC`.

Useful extras

```
SELECT name, score + 5 AS boosted FROM ...;
SELECT UPPER(name), LENGTH(name) FROM students;
SELECT name FROM students
ORDER BY class DESC, name ASC LIMIT 5;
```

Clause order you must write

SELECT → FROM → WHERE → ORDER BY → LIMIT

> Text is in **single quotes** (`'3A'`). Double quotes are for column or table names. In SQLite, `=` on text is **case-sensitive** (`'mei' != 'Mei'`); `LIKE` ignores the case of English letters.

Top mistakes

- Forgetting the `;` (the shell waits with a `...>` prompt).
- Using `==` or `= NULL`.
- Putting `ORDER BY` before `WHERE`.
- `LIKE '%T'` when you wanted names that **start** with T (`'T%'`).